NORTHERN UNIVERSITY OF BUSINESS AND TECHNOLOGY, KHULNA

Department of Computer Science and Engineering

PROJECT PROPOSAL

StudyMate AI

An Adaptive Study Planner and Course Material Assistant for University Students

Week 02 Submission: Initial Project Proposal

Course Code	CSE 4104
Course Title	Software Development III
Team Name	CSE4104-7G-T05
Section	7G

Submitted To	Submitted By
Md. Riaz Mahmud Assistant Professor Department of Computer Science and Engineering Northern University of Business and Technology, Khulna	Md. Samir Ishrak Nafi (Team Leader) - 11230321442 Marshia Hafsa - 11230321502 Zakia Jesmine Jerin - 11230321487 Biplobi Akter Sheme - 11230321493

Table of Contents

Chapter 1: Introduction 1

1.1 Project Title 1

1.2 Project Overview 1

1.3 Target Users 1

1.4 Scope of the Project 1

Chapter 2: Problem Statement 2

2.1 What Problem Currently Exists 2

2.2 Why the Problem Is Important 2

2.3 Why Existing Solutions Are Insufficient 2

2.4 Why AI Is Relevant 2

Chapter 3: Objectives 3

3.1 Primary Objectives 3

3.2 Measurable Success Criteria 3

Chapter 4: Proposed Features 4

4.1 User Roles 4

4.2 Core Features 4

4.3 Optional Stretch Goals 4

4.4 AI Integration Approach 5

4.5 Non-Functional Requirements 5

Chapter 5: Technology Stack 6

5.1 Technologies and Their Purpose 6

5.2 High-Level System Architecture 6

5.3 Reason for Choosing This Stack 6

Chapter 6: Expected Outcome 7

6.1 What Users Will Gain 7

6.2 How the System Improves the Existing Workflow 7

6.3 Expected Benefits of AI Integration 7

Chapter 7: Initial Project Planning 8

7.1 Weekly Plan and Responsibilities 8

7.2 Team Collaboration and Version Control 8

Chapter 8: Risk Analysis 9

Chapter 9: Conclusion 9

References 10

Chapter 1: Introduction

1.1 Project Title

StudyMate AI: An Adaptive Study Planner and Course Material Assistant for University Students

1.2 Project Overview

StudyMate AI is a web application that builds a personal study plan for each university student, adjusts it when the student falls behind, and uses AI to turn the student's own course PDFs into summaries and practice quizzes. The system connects three things that are usually handled separately: the study schedule, the course material, and the student's quiz performance.

This document is the initial project proposal for the semester-long project of CSE 4104 (Software Development III). It describes the problem, objectives, features, technology stack, expected outcomes, project plan and risks of the proposed system.

1.3 Target Users

The primary users are undergraduate university students who study several courses at the same time and must manage chapters, deadlines and exam dates. A secondary role, the Admin, is limited to a simple user-management view and will be added only if time permits.

1.4 Scope of the Project

In scope:

- A web-based study planner with AI-generated and adaptive schedules.
- Upload of text-based course PDFs, AI summaries with page references, and AI-generated quizzes.
- Weak-topic detection from quiz results and a progress dashboard.
- The full development lifecycle: requirements, design, implementation, testing, deployment and documentation.

Out of scope (for the core release):

- OCR for scanned or image-only PDFs.
- Native mobile applications.
- Integration with university learning management systems.

Chapter 2: Problem Statement

2.1 What Problem Currently Exists

University students study several courses at the same time. Each course has its own chapters, deadlines and exam date, and most students plan by memory, in a notebook, or in a calendar they fill in by hand. Course material usually arrives as long PDFs, slides and notes, and turning it into something that can be revised (a summary, a list of key points, practice questions) takes hours. When a student misses a few study sessions the plan stops working, and rebuilding it is usually skipped.

2.2 Why the Problem Is Important

Poor planning leads to last-minute cramming, uneven preparation (comfortable subjects get more time than weak ones), stress, and lower exam results. Time spent building plans and notes by hand is also time not spent learning. A tool that handles this routine work lets students spend their effort on understanding the material.

2.3 Why Existing Solutions Are Insufficient

Based on our initial research, existing tools fall into three groups, and each leaves a gap (Table 2.1).

Table 2.1: Existing tools and their limitations

Type of tool	Examples	Limitation
Calendar and task tools	Google Calendar, Todoist [1], Notion templates	Store tasks but do not decide how much time each subject needs, and do not react when work is missed.
AI schedulers	Motion [2], Reclaim	Built for work meetings and tasks, not for exams and course content.
Document chat tools	ChatPDF, NotebookLM [3]	Answer questions about a file, but do not know the student's exam dates, progress or weak topics.

No single tool we found connects the study plan, the course material and quiz performance. StudyMate AI is designed to fill this gap.

2.4 Why AI Is Relevant

Building a good schedule means balancing difficulty, remaining topics, exam dates and free hours, and this balance changes every time the student misses or finishes work. Fixed rules handle this poorly, while an AI model can reason over these inputs and explain its choices.

Summarising a PDF and writing exam-style questions from it are language tasks that current AI models do well. Quiz results then give real data that can be used to find weak topics and feed them back into the plan. To keep the output reliable, the system will give the AI only the student's own stored data and document text, ask for structured JSON output, and validate it before saving.

Chapter 3: Objectives

3.1 Primary Objectives

1 Build a web-based study planner that generates personalised daily and weekly study schedules from subjects, exam dates, difficulty levels and available study hours.

2 Automatically adapt the schedule when tasks are missed or quiz results show weak topics, without overloading any single day.

3 Use AI to summarise uploaded course PDFs and extract key points, with page references so students can check the source.

4 Generate quizzes from the student's own course material and use the results to detect weak topics.

5 Improve study consistency with a dashboard showing progress, study streak and exam countdown.

6 Deliver a complete, tested, deployed and documented system, developed with regular GitHub commits and a clear division of work.

3.2 Measurable Success Criteria

The targets in Table 3.1 will be used to judge whether the objectives have been met. They are initial targets and may be refined after the requirements phase.

Table 3.1: Success criteria

Objective	Success criterion
Schedule generation	A valid plan is produced for a student with up to 6 subjects, and no day exceeds the student's daily study limit.
Adaptive re-scheduling	Missed tasks are redistributed over upcoming days, and completed history is never changed.
PDF summary	Every key point carries a source page number where the PDF text allows it.
Quiz generation	Each question has options, a correct answer and an explanation, and is linked to a chapter or topic.
Dashboard	Displays today's tasks, subject-wise progress, study streak and exam countdown.
System quality	All core features pass functional, API and UI test cases, and the system is deployed online.

Chapter 4: Proposed Features

4.1 User Roles

- **Student:** can use all the features listed in Section 4.2.
- **Admin:** a simple user-management view, added only if time permits.

4.2 Core Features

Table 4.1 lists the seven core features and shows where AI is used.

Table 4.1: Core features

No.	Feature	Description	Role of AI
1	Authentication and profile	Registration and login (JWT, password hashing); profile and study preferences (daily hours, preferred study time).	No
2	Subject and exam management	Add subjects, chapters and topics; set difficulty, priority and exam dates; exam countdown; topic completion status.	No
3	AI study schedule generator	Creates a daily and weekly plan from subjects, exam dates, difficulty and free hours, with breaks and revision sessions. The student can edit the plan.	Generates the plan as structured JSON
4	Adaptive re-scheduling	Detects missed tasks and redistributes them across upcoming days without overloading any day; gives more time to weak topics.	Re-plans from stored progress data
5	PDF upload and AI summary	Upload course PDFs by subject; get a summary and key points (definitions, concepts, formulas) with page references where possible.	Summarises extracted text
6	AI quiz and weak topic detection	MCQ, short-answer and true/false questions generated from the PDF; automatic scoring, explanations and quiz history; weak topics per chapter.	Writes questions; helps analyse results
7	Dashboard	Today's tasks, completed and pending work, subject-wise progress charts, study streak, exam countdown and AI recommendations.	Writes study recommendations

4.3 Optional Stretch Goals

These will be attempted only if the core features are stable, tested and deployed.

- Ask-Your-PDF chatbot with page references
- Flashcards generated from uploaded material
- Bangla language support for AI summaries and explanations
- Admin panel with usage statistics
- OCR support for scanned PDFs

4.4 AI Integration Approach

Every AI feature follows the same flow: user input goes to the backend, which sends only the relevant stored data to the Gemini API [7]; the structured response is checked by the backend, saved in the database, and shown in the frontend. The AI key stays on the server and is never exposed to the browser. Table 4.2 gives the details for each feature.

Table 4.2: AI integration for each feature

Feature	Data sent to AI	AI returns	What the system does
Schedule generation	Subjects, topics, difficulty, exam dates, daily hours, completion status	List of study sessions (date, topic, duration, type)	Checks that no day exceeds the daily limit, then saves the sessions as tasks
Adaptive re-planning	Missed and remaining tasks, days left, weak topics	Updated list of future sessions	Replaces future tasks and keeps completed history unchanged
PDF summary	Extracted text of the chosen chapter or page range	Summary and key points with page numbers	Stores and shows them with the source page
Quiz generation	Extracted text of the chosen chapter	Questions, options, correct answer, explanation	Stores the quiz; scoring and weak-topic counts are done by backend code

Scoring and weak-topic counting are deliberately done by ordinary backend code rather than by the AI, so that results are consistent and can be checked.

4.5 Non-Functional Requirements

- **Security:** passwords are hashed with bcrypt, sessions use JSON Web Tokens [8], and API keys are stored only as server-side environment variables.
- **Reliability:** all AI output is validated against a fixed JSON structure before it is saved; invalid output is rejected and the request is retried or reported to the user.
- **Performance:** only the needed text is sent to the AI and results are cached to reduce delay and API usage.
- **Usability:** a clean, responsive interface that works on both desktop and mobile browsers.
- **Maintainability:** modular code, version control on GitHub, and API documentation.

Chapter 5: Technology Stack

5.1 Technologies and Their Purpose

Table 5.1: Technology stack

Layer	Technology	Purpose
Frontend	React.js [4], Tailwind CSS, Recharts	User interface, dashboard and charts
Backend	Node.js [5] with Express.js	REST APIs, JWT authentication, bcrypt, file upload, PDF text extraction (pdf-parse)
Database	PostgreSQL (Supabase [6])	Users, subjects, topics, tasks, documents, quizzes and results
AI Integration	Gemini API [7] (OpenAI API as an alternative)	Schedule generation, re-planning, summaries and quiz questions
Deployment	Vercel, Render, Supabase	Frontend, backend and database hosting
Tools	GitHub, Figma, Postman, draw.io, dbdiagram.io	Version control, UI design, API testing, diagrams

5.2 High-Level System Architecture

The system uses a three-tier architecture with a separate AI service layer:

1 **Frontend (React):** the student interacts with pages for planning, documents, quizzes and the dashboard.

2 **Backend (Express REST API):** handles authentication, business rules, PDF text extraction, prompt preparation and validation of AI responses.

3 **Database (PostgreSQL on Supabase):** stores all user data, schedules, documents, quizzes and results.

4 **AI service (Gemini API):** called only from the backend with the relevant stored data.

The detailed architecture diagram will be prepared in the system design phase (Week 4).

5.3 Reason for Choosing This Stack

- React, Node.js and PostgreSQL are widely used, well documented and match the skills covered in the course.
- Supabase, Vercel and Render offer free tiers suitable for a student project.
- The Gemini API supports structured output and has a free tier; the OpenAI API is kept as a fallback if limits or availability become a problem.

Chapter 6: Expected Outcome

6.1 What Users Will Gain

- A ready-made study plan for every subject, based on their own exam dates and free time.
- Quick summaries, key points and practice quizzes from their own course PDFs.
- A clear view of progress, weak topics and days left before each exam.

6.2 How the System Improves the Existing Workflow

Today a student plans by hand, rewrites the plan when things go wrong, and makes notes and questions manually. With StudyMate AI the plan is created in minutes, updates itself when tasks are missed, and the course material, quizzes and progress are kept in one place.

6.3 Expected Benefits of AI Integration

- Plans that adapt to each student instead of one fixed template.
- Less time spent on routine preparation work, more time spent studying.
- Weak topics found from real quiz results and fed back into the schedule.
- Answers and summaries based on the student's own material, with page references.

Chapter 7: Initial Project Planning

7.1 Weekly Plan and Responsibilities

The plan follows the course weekly workflow (Table 7.1). Responsibilities are divided by role, and every member will push work to GitHub every week.

Table 7.1: Weekly project plan

Week	Activity	Responsible
3	SRS: requirements, user roles, use cases	All members (leader coordinates)
4	System design: architecture, ER diagram, use case diagram, API structure	Marshia (ER), Biplobi (APIs), Samir (architecture)
5	UI/UX wireframes in Figma	Zakia
6	Backend: authentication, database, core REST APIs	Biplobi, Marshia
7	Frontend: pages, dashboard, API integration	Zakia
8	AI integration: schedule, re-planning, summary, quiz	Samir, with Biplobi
9	Feature completion: quizzes, weak topics, dashboard polish	All members
10	Testing and debugging (functional, API, UI, AI)	All members
11	Deployment (Vercel, Render, Supabase)	Biplobi, Samir
12	Final report, user manual, API documentation	All members
13	Presentation slides and demo preparation	All members
14	Final presentation and viva	All members

7.2 Team Collaboration and Version Control

- All code is kept in a shared GitHub repository, and each member commits their work every week.
- Work is divided by feature or layer, and the team leader coordinates progress and merges.
- Progress is reviewed at the end of each week against the plan above.

Chapter 8: Risk Analysis

Table 8.1 lists the main risks identified so far and how the team plans to handle them.

Table 8.1: Risks and mitigation

Risk	Plan to handle it
AI output is wrong or badly formatted	Request structured JSON, validate it in the backend, and keep the student able to edit the plan.
Free AI quota or rate limits	Send only the needed text, cache results, and limit the size of each request.
PDFs that are scanned or poorly formatted	Support text-based PDFs in the core scope; OCR is left as a stretch goal.
Too many features for one semester	Finish the seven core features first; stretch goals only after testing and deployment.

Chapter 9: Conclusion

StudyMate AI addresses a real and common problem: students plan manually, fall behind, and spend hours converting course material into revision notes. By combining an adaptive study planner, PDF summaries and quizzes in one system, and by using AI only on the student's own validated data, the project offers a practical and reliable solution. The scope is limited to seven core features, the technology stack is well suited to the course, and the weekly plan gives a clear path from requirements to final presentation.

References

[1] Doist, "Todoist: To-do list and task manager." [Online]. Available: https://todoist.com. Accessed: Nov. 9, 2026.

[2] Motion, "Motion: AI calendar and project manager." [Online]. Available: https://www.usemotion.com. Accessed: Nov. 9, 2026.

[3] Google, "NotebookLM." [Online]. Available: https://notebooklm.google. Accessed: Nov. 9, 2026.

[4] Meta Open Source, "React documentation." [Online]. Available: https://react.dev. Accessed: Nov. 9, 2026.

[5] OpenJS Foundation, "Node.js." [Online]. Available: https://nodejs.org. Accessed: Nov. 9, 2026.

[6] Supabase, "Supabase documentation." [Online]. Available: https://supabase.com/docs. Accessed: Nov. 9, 2026.

[7] Google, "Gemini API documentation." [Online]. Available: https://ai.google.dev/gemini-api/docs. Accessed: Nov. 9, 2026.

[8] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519